

Módulo 05 — Sistemas multiagênticos e MCP

Padrões para organizar times de agentes, quando isso é exagero, e o Model Context Protocol: host, cliente, servidor, tools, resources e prompts.

1. Visão geral
2. Conceitos
3. Exercício
4. Projeto final — etapa M5
5. Para revisar

Visão geral

Objetivos de aprendizagem

Ao final, a pessoa consegue:

1. reconhecer os padrões multiagente (pipeline, supervisor, hierárquico, handoff, gerador + revisor) e dar um exemplo de cada;
2. decidir, com critérios, quando dividir um trabalho em vários agentes é exagero;
3. explicar o MCP: host, cliente e servidor; tools, resources e prompts; transportes stdio e HTTP;
4. escrever um servidor MCP local, testá-lo com um cliente MCP e conectar as tools dele a um agente;
5. aplicar menor privilégio e validação de entrada a ferramentas expostas por MCP.

Mensagem central

Divida em vários agentes só quando as partes forem **independentes**. E ligue todos ao mundo por um **protocolo padrão**: o servidor MCP não precisa saber quem vai usá-lo, e o agente não precisa saber como cada ferramenta foi feita.

Conceitos

No Módulo 4, o curador virou **um** agente. Este módulo faz duas perguntas:

- 1. **E se fossem vários?** Um time de agentes, cada um com um papel (coletor, classificador, redator, revisor). Quais formas de organizar um time existem e quando isso é exagero?
- 2. **Como os agentes acessam o mundo de um jeito padrão?** O **MCP** (*Model Context Protocol*): um protocolo aberto para expor ferramentas, dados e prompts a qualquer aplicação com LLM.

1. Padrões multiagente

"Multiagente" é qualquer sistema em que mais de um agente (ou papel de LLM) trabalha na mesma tarefa. A pergunta que importa é **como eles se ligam**:

Padrão	Como funciona	Exemplo	Quando faz sentido
Pipeline	cada agente faz uma etapa e passa o resultado para o próximo	coletor → classificador → redator → revisor	etapas claras, em ordem fixa
Supervisor (orquestrador-trabalhadores)	um agente central decide, a cada rodada, qual trabalhador chamar	um pesquisador-chefe que distribui subtarefas	as subtarefas variam com o pedido
Hierárquico	supervisores de supervisores: cada um cuida de uma equipe	um gerente com times de pesquisa e de redação	problemas grandes, com subproblemas que também se dividem
Handoff (transferência)	o agente que está atendendo passa a conversa para outro mais adequado	triagem → financeiro → técnico, numa central de atendimento	conversas em que a especialidade muda no caminho
Gerador + revisor (avaliador-otimizador)	um gera, outro avalia e devolve críticas, até aprovar ou desistir	redator ⇌ revisor	há um critério de qualidade claro para checar

Analogia: é uma cozinha de restaurante. Na **linha de montagem** (pipeline), cada cozinheiro faz uma parte do prato. O **chef** (supervisor) decide quem faz o quê conforme o pedido. Numa cozinha grande, há **chefs de praça** abaixo do chef (hierárquico). Quando um cliente pede algo que só o confeitoiro sabe, o garçom **passa** o pedido (handoff). E o chef **prova** e devolve o prato até ficar bom (gerador + revisor).

No projeto, a orquestração do time é um **pipeline em código** (um workflow, no sentido do Módulo 4), com um ciclo **gerador + revisor** entre redator e revisor. Só o **coletor** é um agente de verdade, escolhendo quais fontes ler. Os outros papéis são chamadas de LLM com uma tarefa só.

Quando multiagente é exagero

Duas visões, publicadas com um dia de diferença em junho de 2025, ajudam a calibrar:

- **A favor, com custos claros.** A Anthropic descreveu o seu sistema de pesquisa com um agente-líder que dispara subagentes em paralelo ([How we built our multi-agent research system, 2025](#)). Nas avaliações internas deles, o sistema multiagente superou um único agente em 90,2%, mas usou cerca de **15x mais tokens** que um chat (um agente sozinho usa cerca de 4x). Eles apontam que **não** é um bom encaixe para tarefas em que todos os agentes precisam do mesmo contexto ou que têm muitas dependências entre si, como a maioria das tarefas de programação.
- **Contra, como padrão.** A Cognition argumentou o oposto ([Don't Build Multi-Agents, 2025](#)): agentes em paralelo, sem o contexto completo uns dos outros, tomam **decisões conflitantes**. Os dois princípios propostos são: compartilhe o contexto inteiro (o histórico dos agentes, não só mensagens soltas) e lembre que **ações carregam decisões implícitas**. A recomendação deles é um agente único, em sequência, como padrão.

Os dois concordam no essencial: **multiagente compensa quando o trabalho se divide em partes independentes que podem rodar em paralelo**. Quando as partes dependem umas das outras, dividir cria conflitos e custo.

Perguntas antes de dividir em vários agentes:

1. As partes são **independentes**? Dá para rodar em paralelo?
2. Cada papel precisa de **contexto, ferramentas ou permissões diferentes**? (Separar o que lê de fora do que publica é uma boa razão.)
3. O ganho paga o **custo** extra (tokens, latência, depuração)?
4. Um **pipeline em código** com chamadas simples não resolve? (Quase sempre é o primeiro passo.)

2. MCP: Model Context Protocol

O problema

Cada aplicação com LLM (um chat, um editor de código, o nosso curador) precisa se conectar a dados e ferramentas: arquivos, bancos, APIs, feeds. Sem um padrão, cada par aplicação x integração vira um conector diferente.

O **MCP** é um protocolo aberto que padroniza essa conexão. A inspiração declarada é o *Language Server Protocol*, que padronizou como editores de código dão suporte a linguagens. **Analogia:** uma tomada universal. Quem faz o aparelho (o servidor MCP) não precisa saber em que casa (aplicação) ele vai ser ligado.

Os papéis

- **Host:** a aplicação com LLM (um chat, um editor, o nosso time de agentes).
- **Cliente:** o conector, dentro do host, que fala com **um** servidor.
- **Servidor:** o serviço que oferece contexto e capacidades (o nosso servidor de fontes, o de edição, o glossário).

As mensagens são **JSON-RPC 2.0**: pedidos com `method` e `params`, respostas com `result` ou `error`. A `demo:protocolo` mostra as mensagens reais.

O que um servidor oferece

Recurso	Quem controla	Para que	No projeto
Tools (ferramentas)	o modelo decide chamar	funções que o modelo executa	<code>listar_fontes</code> , <code>ler_fonte</code> , <code>publicar_edicao</code>
Resources (recursos)	a aplicação decide ler	dados e contexto, por URI	<code>noticia://{id}</code> , <code>edicao://ultima</code>
Prompts	o usuário escolhe usar	mensagens prontas, com parâmetros	<code>classificar_noticia</code> , <code>explicar_termo</code>

Ferramentas podem declarar **anotações** como `readOnlyHint` (só lê), `destructiveHint` (apaga ou altera) e `idempotentHint` (repetir não muda o resultado). São **dicas**: a especificação manda tratá-las como não confiáveis, a menos que o servidor seja confiável.

Do lado do cliente, a revisão atual mantém a **elicitation**: o servidor pode pedir ao usuário uma informação a mais no meio de uma tarefa.

Transportes

- **stdio:** o cliente inicia o servidor como um subprocesso e troca mensagens pela entrada e saída padrão (uma linha JSON por mensagem). É o mais simples para servidores locais. Por isso **o servidor não pode escrever nada em `stdout`**: esse é o canal do protocolo. Logs vão para `stderr`.
- **Streamable HTTP:** cada mensagem é um POST HTTP para um endpoint MCP; a resposta vem como JSON ou por um fluxo SSE. É o caminho para servidores remotos.

Duas eras do protocolo (atenção)

A especificação atual é a revisão **2026-07-28** (**especificação**). A principal mudança em relação à anterior (2025-11-25) é que as requisições ficaram **sem estado**: cada uma carrega a versão do protocolo e as capacidades do cliente no próprio corpo (`_meta`), e acabou o *handshake* `initialize`. As revisões anteriores tinham uma sessão aberta com `initialize`.

Na prática, os dois mundos convivem. Com o SDK oficial de TypeScript (v2):

- o **cliente** usa a era antiga por padrão (`versionNegotiation: 'legacy'`); em modo `'auto'`, ele sonda a revisão nova e volta para a antiga se o servidor não suportar;
- o **servidor** atende as duas eras quando servido por `serveStdio(...)` (stdio) ou `createMcpHandler(...)` (HTTP); conectado direto a um transporte com `connect()`, como nos nossos testes em memória, ele fala a era antiga.

A `demo:protocolo` mostra as duas: em memória, com `initialize` (2025-11-25); por stdio, em modo `auto`, sem `initialize` e com `protocolVersion 2026-07-28` em cada requisição. Para o que fazemos no curso (tools, resources, prompts), o comportamento é o mesmo nas duas eras.

Segurança

MCP dá ao modelo **acesso a dados e execução de código**. Os princípios da especificação: consentimento e controle do usuário, privacidade dos dados e cuidado com ferramentas ("ferramentas representam execução arbitrária de código"). Na prática:

- **menor privilégio**: cada agente recebe só as ferramentas de que precisa (no projeto, o coletor só recebe `listar_fontes` e `ler_fonte`, filtradas no adaptador);
- **valide toda entrada que vem do modelo**: `ler_fonte` só lê o arquivo depois de checar que a fonte existe (nunca monte um caminho de arquivo com texto do modelo sem checar);
- **o que um servidor devolve é dado não confiável**: pode trazer prompt injection (Módulo 3);
- **ferramentas que mudam algo** pedem aprovação humana (Módulo 7);
- só instale servidores MCP de fontes confiáveis.

Do MCP para um agente

Um agente (o loop do Módulo 4) não "fala MCP" sozinho: alguém precisa listar as tools do servidor e transformá-las no formato de ferramenta do agente. No projeto, `ferramentasDoMcp` faz isso: para cada tool do servidor, cria uma ferramenta do AI SDK com o mesmo nome, descrição e schema, cuja execução chama `callTool` no servidor. O que importa: o mesmo servidor serve o nosso time, um agente de código ou o MCP Inspector, sem mudar uma linha.

Referências

- Model Context Protocol — especificação (revisão 2026-07-28):
<https://modelcontextprotocol.io/specification/2026-07-28> · tools:
<https://modelcontextprotocol.io/specification/2026-07-28/server/tools>
- SDK oficial de TypeScript (v2): <https://ts.sdk.modelcontextprotocol.io/v2/>
- MCP Inspector (ferramenta para testar servidores):
<https://github.com/modelcontextprotocol/inspector>
- Anthropic (2025). *How we built our multi-agent research system*.
<https://www.anthropic.com/engineering/multi-agent-research-system>
- Cognition (2025). *Don't Build Multi-Agents*. <https://cognition.com/blog/dont-build-multi-agents>
- Anthropic (2024). *Building effective agents* (padrões de workflow, incluindo orquestrador-trabalhadores e avaliador-otimizador). <https://www.anthropic.com/engineering/building-effective-agents>

Exercício

Tarefa assíncrona. Os testes rodam offline: não precisa de modelo rodando.

Duas peças:

1. **um servidor MCP de verdade:** o glossário de IA da mentoria, com tools, um resource e um prompt. Os testes conectam um cliente MCP real a ele (em memória) e usam o que ele oferece;
2. **os padrões multiagente como funções:** pipeline, supervisor e gerador + revisor. Os "agentes" são funções de mentira nos testes, porque o que importa aqui é a forma de ligar os agentes.

Como rodar

Da raiz do repositório:

```
pnpm -F @mentoria/ex05-mcp test          # versão base
pnpm -F @mentoria/ex05-mcp test:desafio   # desafio extra
```

Os testes **falham** até você implementar. A solução de referência está em [solucao/](#). Tente antes de abrir. `pnpm -F @mentoria/ex05-mcp test:solucao` roda os mesmos testes contra ela.

Versão base

Arquivo	O que fazer
<code>src/glossario/servidor.ts</code>	<code>criarServidorGlossario</code> : tools <code>buscar_termo</code> e <code>listar_termos</code> (só leitura), resource <code>glossario://{termo}</code> e prompt <code>explicar_termo</code>
<code>src/padroes.ts</code>	<code>emPipeline</code> , <code>comSupervisor</code> e <code>comRevisao</code>

Fornecidos: os termos (`src/glossario/termos.ts`), `conectarEmMemoria` e `normalizar` (`src/glossario/conectar.ts`).

Dicas:

- Use o SDK oficial v2: `McpServer` e `ResourceTemplate` de `@modelcontextprotocol/server`; schemas com Zod. Veja o servidor de fontes do projeto final, ou a [documentação](#).

- Erro de ferramenta **não** é exceção: devolva `{ isError: true, content: [...] }`. O cliente (e o modelo) leem a mensagem.
- Em `comSupervisor`, quem decide é a função `escolher`; o seu código só executa, registra e para nos limites.

Feito = `pnpm -F @mentoria/ex05-mcp test` com todos os testes passando.

Use de verdade (opcional)

Conecte o seu servidor a um cliente MCP externo, o [MCP Inspector](#):

```
npx @modelcontextprotocol/inspector pnpm -F @mentoria/ex05-mcp glossario
```

Ele abre uma página no navegador em que você lista as tools, chama `buscar_termo`, lê `glossario://agente` e testa o prompt. Se você usa um agente de código compatível com MCP, pode registrar o mesmo comando como servidor local e perguntar a ele sobre os termos da mentoria.

Desafio extra — handoff

Em `src/desafio/handoff.ts`, implemente `executarHandoffs`: cada agente responde ou transfere a conversa para outro, como numa central de atendimento. O loop precisa parar em agente desconhecido, em **ping-pong** (voltar para quem já atendeu) e num limite de transferências.

Feito = `pnpm -F @mentoria/ex05-mcp test:desafio` com todos os testes passando.

Perguntas para pensar

1. Por que `buscar_termo` é uma tool e `glossario://{termo}` é um resource, se os dois devolvem a mesma informação? Quem decide usar cada um?
2. O que acontece se um servidor MCP escrever um log com `console.log` quando roda por stdio?
3. No seu `comSupervisor`, o que impede o supervisor de chamar o mesmo trabalhador para sempre?
4. Pense num processo do seu time que hoje passa por várias pessoas. Qual padrão (pipeline, supervisor, handoff, gerador + revisor) ele lembra? Faria sentido com agentes?

Projeto final — etapa M5

Etapa M5 — o time de agentes e os servidores MCP

Objetivo: o curador vira um **time** com quatro papéis (coletor, classificador, redator, revisor), que acessa as fontes e publica a edição por **MCP**. As fontes agora são feeds RSS (`dados/rss/`).

Implemente em `src/m05/`:

1. **servidor-fontes.ts**: servidor MCP com as tools `listar_fontes` e `ler_fonte` (lê o RSS), o resource `noticia://{id}` e o prompt `classificar_noticia`. Nunca leia um arquivo cujo nome veio do modelo sem checar se a fonte existe.
2. **servidor-edicao.ts**: `montarMarkdown` e o servidor com a tool `publicar_edicao` (grava `edicao.md`, recusa item sem fonte `https`) e o resource `edicao://ultima`.
3. **mcp-para-ai-sdk.ts**: `ferramentasDoMcp`, que transforma as tools de um servidor MCP em ferramentas do AI SDK, com lista de permitidas (menor privilégio).
4. **time.ts**: `montarEdicao`, a orquestração: coletar → classificar → os 10 publicáveis de maior confiança → redigir ⇌ revisar (até 2 tentativas, com o motivo da reprovação) → publicar.

Fornecidos: `rss.ts` (leitor de RSS) e `papeis.ts` (os quatro papéis com LLM: o coletor é um agente com as tools do MCP de fontes; o revisor checa por código antes de chamar o juiz).

Feito =

1. `pnpm -F @mentoria/curador exec vitest run test/m05.test.ts verde` (offline; os servidores são testados por um cliente MCP em memória);
2. o time roda com o modelo de verdade e publica a edição:

```
pnpm -F @mentoria/curador m05:edicao      # grava saidas/edicao.md e saidas/m05-  
relatorio.md
```

Abra a edição e o relatório e responda (vamos discutir na Aula 6):

- Quanto tempo e quantas chamadas cada papel gastou?
- O coletor deixou alguma notícia boa para trás?
- O revisor reprovou algo? A segunda tentativa do redator resolveu?
- Este problema precisava de vários agentes? O que o time ganhou e o que custou em relação ao pipeline da M4?

Desafio extra: conecte o servidor de fontes ou o de edição a um cliente MCP externo:

```
npx @modelcontextprotocol/inspector pnpm -F @mentoria/curador mcp:fontes
```

Desafio extra opcional (integração externa): escreva um servidor de edição alternativo que publique no Notion, no Slack ou por e-mail, com a mesma tool `publicar_edicao`. O time não muda uma linha: só o servidor de destino. (Exige token do serviço; não é necessário para concluir a etapa.)

Para revisar

Perguntas para discussão (15 min)

1. O curador precisava de quatro agentes? O que o time ganhou em relação ao pipeline da Aula 4, e o que custou?
2. Um processo do seu time lembra qual padrão: pipeline, supervisor, handoff ou gerador + revisor? Faria sentido com agentes?
3. Por que `buscar_termo` é uma tool e `glossario://{termo}` é um resource, se devolvem a mesma informação?
4. Que servidor MCP você gostaria de ter no seu trabalho? Quais tools ele teria, e quais seriam só leitura?
5. O que pode dar errado ao instalar um servidor MCP de uma fonte desconhecida?

Armadilhas comuns

- **Dividir em vários agentes o que é sequencial e dependente.** Vira custo e decisões conflitantes.
- **Chamar de "multiagente" um pipeline com várias chamadas de LLM.** Tudo bem ser um pipeline; é mais simples de testar.
- **`console.log` num servidor stdio:** quebra o protocolo, porque stdout é o canal das mensagens. Use stderr.
- **Montar caminho de arquivo com texto vindo do modelo** sem checar (ler `../../../../etc/passwd`).
- **Dar todas as tools a todos os agentes.** Use lista de permitidas.
- **Confiar nas anotações (`readOnlyHint`) de um servidor desconhecido.** São dicas, não garantias.
- **Lançar exceção para erro de ferramenta.** Devolva `isError` com a mensagem, para o modelo se corrigir.
- **Instalar servidores MCP sem saber de onde vêm.** É código rodando com o seu acesso.